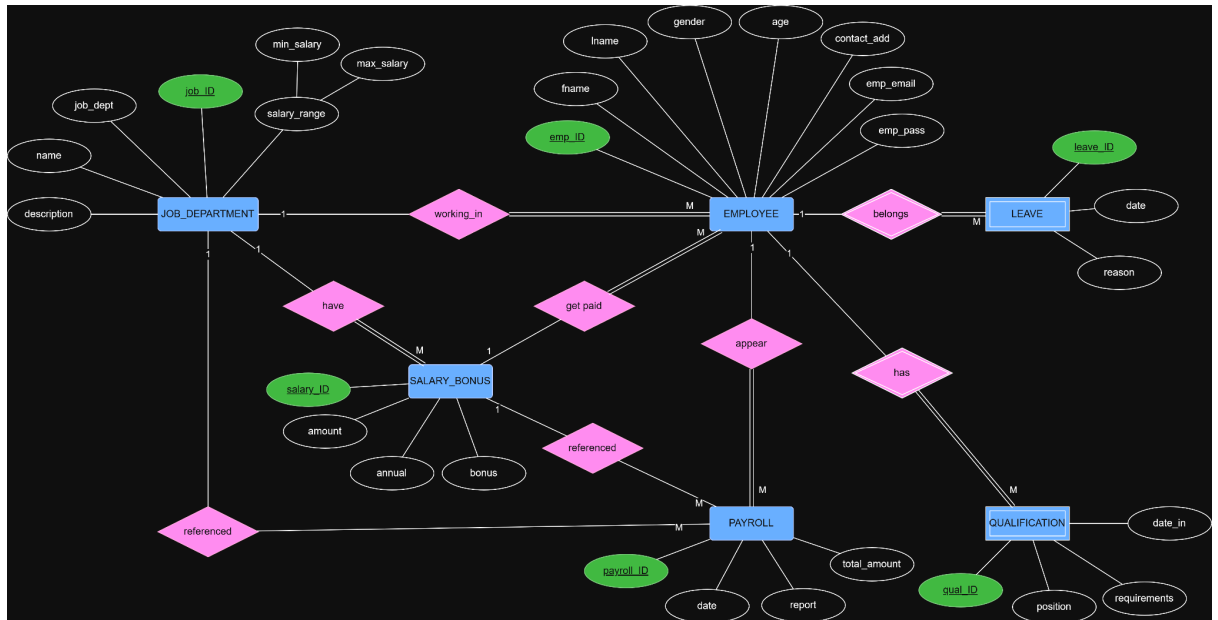


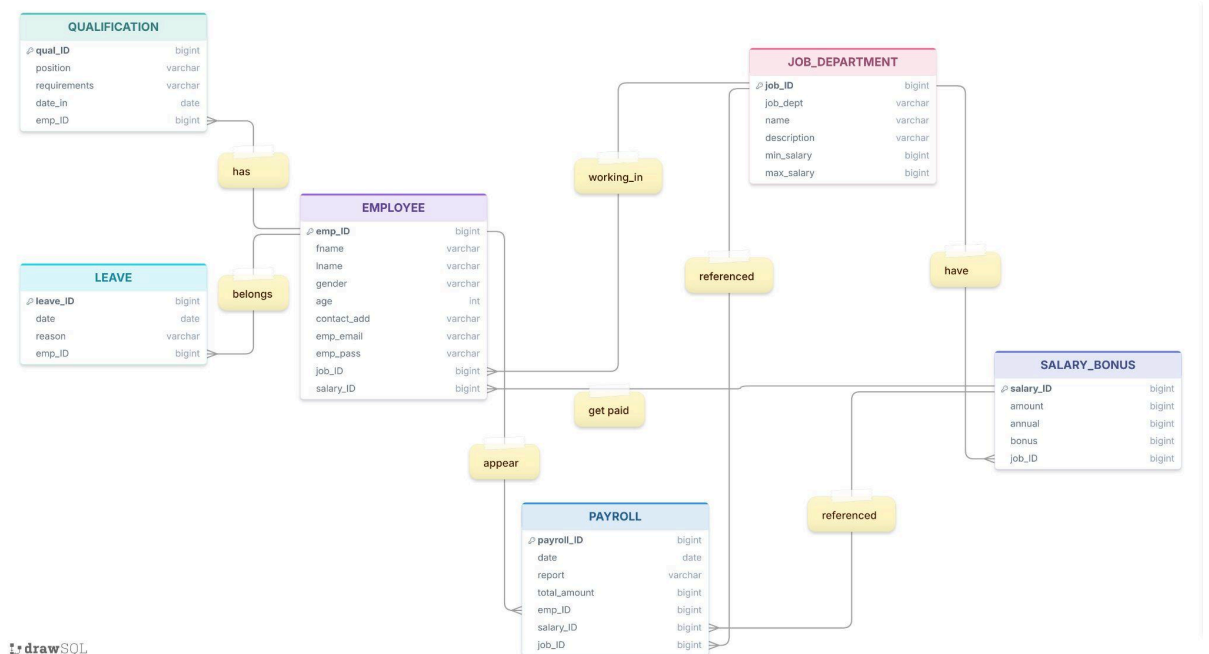
EMPLOYEE MANAGEMENT SYSTEM

❖ ERD Design & Mapping

ERD Design:



Mapping:



❖ Normalization

Task 1: First Normal Form (1NF) Analysis:

(a) Violations of 1NF in EMP_RAW

The EMP_RAW table violates 1NF in the following ways:

1. phone_numbers column - Contains multiple values (multiple phone numbers) in a single cell. 1NF requires atomic (indivisible) values in each column.
2. job_titles column - Contains multiple job titles for a single employee, violating atomicity requirements.
3. full_name column - Though technically could be considered atomic, it combines first name and last name. While not a strict 1NF violation, it's poor design for querying individual name parts.
4. dept_name and dept_location - These columns contain repeating groups, as an employee can belong to multiple departments with different locations.
5. salary column - Without proper normalization, this could contain multiple salary values if an employee has multiple job titles with different salaries.

(b) EMP_RAW in 1NF

To normalize to 1NF, we need to:

- Separate multi-valued attributes into individual rows
- Create separate tables for related entities

Resulting tables in 1NF:

EMPLOYEE_1NF (Base employee info)

emp_id	first_name	last_name	salary
E001	John	Smith	65000
E002	Mary	Johnson	72000
E003	Robert	Williams	58000

EMPLOYEE_PHONE (Phone numbers)

emp_id	phone_number
E001	555-0101
E001	555-0102
E002	555-0201
E003	555-0301
E003	555-0302
E003	555-0303

EMPLOYEE_DEPT_ASSIGNMENT (Department assignments)

emp_id	dept_name	dept_location
E001	IT	Building A
E001	HR	Building B
E002	Finance	Building C
E003	Marketing	Building D
E003	Sales	Building E

EMPLOYEE_JOB_TITLES (Job titles)

emp_id	job_title
E001	Developer
E001	Team Lead
E002	Accountant
E003	Marketing Manager

(c) 1NF Compliance of Your EMS Schema

All 6 tables in EMS schema satisfy 1NF because:

JOB_DEPARTMENT

- ✓ All columns contain atomic values
- ✓ No repeating groups
- ✓ Each column has a single value per row
- ✓ job_ID serves as primary key

SALARY_BONUS

- ✓ Atomic values in all columns
- ✓ No repeating groups
- ✓ Single value per column per row
- ✓ salary_ID is primary key, job_ID is foreign key

QUALIFICATION

- ✓ Atomic values (although "requirements" could contain multiple items, as a text field it's considered atomic)
- ✓ No multi-valued attributes
- ✓ Single value per column per row
- ✓ emp_ID serves as foreign key

EMPLOYEE

- ✓ All columns contain atomic values
- ✓ No repeating groups
- ✓ Each column has a single value
- ✓ Proper primary and foreign keys

PAYROLL

- ✓ Atomic values in all columns
- ✓ No repeating groups
- ✓ Single value per row per column
- ✓ Proper foreign key relationships

LEAVE

- ✓ Atomic values
- ✓ No repeating groups
- ✓ Single value per column
- ✓ emp_ID is foreign key

Key Observations:

1. The schema properly separates related entities into different tables
2. Multi-valued attributes have been eliminated through proper table design
3. Each table has a clear primary key
4. Relationships are maintained through foreign keys
5. All columns contain atomic, indivisible values

Task 2: Second Normal Form (2NF) Analysis:

(a) Partial Dependencies in PAYROLL_RAW

With composite PK (emp_ID, job_ID), let's identify what depends on the whole key vs. parts:

Full key dependency (emp_ID + job_ID):

- pay_date - depends on both employee AND job
- total_amount - depends on both employee AND job

Partial dependencies (depend on only PART of the key):

1. emp_name → depends ONLY on emp_ID (not on job_ID)
 - a. An employee's name is determined by their ID, regardless of which job they're performing
2. job_title → depends ONLY on job_ID (not on emp_ID)
 - a. A job title is determined by the job ID, regardless of which employee holds it
3. salary_range → depends ONLY on job_ID (not on emp_ID)
 - a. The salary range is associated with the job position, not the individual employee

(b) Decomposition into 2NF

To achieve 2NF, we need to remove partial dependencies by separating them into their own tables:

Table 1: EMPLOYEE_INFO (emp_ID as PK, emp_name)

emp_ID	emp_name
E001	John Smith
E002	Mary Johnson
E003	Robert Williams

Table 2: JOB_INFO (job_ID as PK, job_title, salary_range)

job_ID	job_title	salary_range
J001	Developer	50,000-70,000
J002	Accountant	45,000-60,000
J003	Manager	60,000-85,000

Table 3: PAYROLL_TRANSACTION (emp_ID, job_ID as composite PK, pay_date, total_amount)

emp_ID	job_ID	pay_date	total_amount
E001	J001	2026-01-15	5,200
E001	J002	2026-01-15	3,800
E002	J002	2026-01-15	4,500
E003	J003	2026-01-15	6,500

Foreign Key Relationships:

- PAYROLL_TRANSACTION.emp_ID references EMPLOYEE_INFO.emp_ID
- PAYROLL_TRANSACTION.job_ID references JOB_INFO.job_ID

(c) Review of Your EMS Schema for 2NF

Analysis of your 6 tables:

Table	Composite PK?	Partial Dependencies?	2NF Compliant?
JOB_DEPARTMENT	No	N/A	Yes
SALARY_BONUS	No	N/A	Yes
QUALIFICATION	No	N/A	Yes
EMPLOYEE	No	N/A	Yes
PAYROLL	No	N/A	Yes
LEAVE	No	N/A	Yes

Justification:

None of the tables have composite primary keys. All six tables use single-attribute primary keys:

Conclusion:

- Since no table has a composite primary key, there cannot be any partial dependencies
- Partial dependencies only exist when you have composite keys where some attributes depend on part of the key
- The schema is already in 2NF by default because all tables have single-attribute primary keys

❖ SQL DML — INSERT / UPDATE / DELETE / SELECT

Solution for Task 1 in file named: 03_ddl+dml.sql

Solution for Task 2 in file named: 03_dql.sql

❖ Aggregation Functions

Solution for Task 1&2 in file named: 04_aggregation.sql

❖ Joins

Solution for Task 1&2 in file named: 05_joins.sql

❖ Subqueries

Solution for Task 1&2 in file named: 06_subqueries.sql

❖ Views

Solution for Task 1&2 in file named: 07_views.sql

❖ Stored Procedures & Functions

Solution for Task 1&2 in file named: 08_procedures.sql

❖ Triggers

Solution for Task 1&2 in file named: 09_triggers.sql

EMP_ID	Full Name	EMP_EMAIL
1	12 Auto Assign	auto.assign@company.com

LOG_ID	EMP_ID	ACTION	LOG_TIMESTAMP
1	2	14 NEW HIRE	2026-06-20 13:57:10
2	1	13 NEW HIRE	2026-06-20 13:57:10

TRIGGER_NAME	TRIGGER_TYPE	TRIGGERING_EVENT	TABLE_NAME	STATUS
1 TRG_EMP_ID	BEFORE EACH ROW	INSERT	EMPLOYEE	ENABLED
2 TRG_EMP_WELCOME_LOG	AFTER EACH ROW	INSERT	EMPLOYEE	ENABLED

❖ Oracle Scheduler Jobs

Solution for Task 1&2 in file named: 10_scheduler.sql